LSM Tree Storage Engine

Technical project brief for software engineering internships

Brad Asher | Rust | github.com/BradshawAsher/lsm-tree

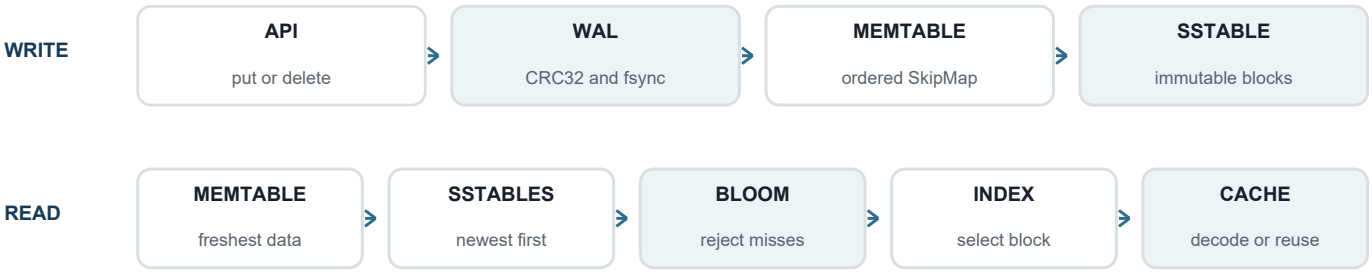
Project overview

I built an embedded key-value storage engine in Rust to understand how databases turn writes in memory into durable, searchable files on disk. The engine exposes put, get, delete, and range-scan operations while preserving data across restarts. Its design combines a checksummed write-ahead log, an ordered concurrent MemTable, immutable SSTables, Bloom filters, an LRU block cache, a K-way merge iterator, and full merge compaction.

The project focuses on correctness at storage boundaries: a write is synchronized to the log before it enters memory; immutable files carry explicit indexes and metadata; newer values mask older versions; and tombstones preserve delete semantics until compaction can safely reclaim them.

Rust 2024 library and CLI	**16 of 16** tests passing	**4 KB** target data blocks	**CRC32 and fsync** durable WAL path

Architecture



Core design

COMPONENT	IMPLEMENTATION AND PURPOSE
Write ahead log	Each record stores a CRC32 checksum, key and value lengths, an operation code, and payload bytes in a deterministic big-endian frame. The engine calls fsync before acknowledging the write and replays valid records at startup.
MemTable	A crossbeam SkipMap keeps keys ordered and supports concurrent access. A short RwLock protects the active Arc during freeze and swap; the log writer remains serialized because writes must reach disk in order.
SSTables	Flushes create immutable files with packed data blocks, a sparse first-key index, a serialized Bloom filter, and a fixed 40-byte footer. Point lookups use the filter and index before reading or reusing one decoded block.
Version resolution	Reads inspect memory first and then SSTables from newest to oldest. Range scans merge sorted sources with a min-heap, retain the newest copy of each key, and suppress tombstones.

Engineering decisions and tradeoffs

I treated the engine as a correctness-first prototype. The choices below made failure behavior and version ordering explicit, while also exposing the next performance work.

DECISION	WHY IT WORKS	CURRENT COST
Sync every WAL record	Acknowledged writes survive a normal process restart and WAL corruption is detected before replay.	One fsync per operation limits write throughput. Group commit would amortize the cost.
Immutable SSTables	Flushes remain sequential and readers never observe an in-place disk update.	Old versions and tombstones consume space until compaction.
Full merge compaction	A newest-first merge has simple, testable precedence rules and can purge bottom-level tombstones.	The current pass is synchronous and rewrites every table. Leveled compaction would bound work.
Materialized scan results	The public API is straightforward and the heap merge keeps output sorted with O(log K) source selection.	Large scans allocate a result vector. A streaming iterator would reduce peak memory.

Correctness and verification

The automated suite currently passes 16 of 16 tests. It covers WAL replay and checksum failures, ordered MemTable behavior, block encoding and binary search, Bloom-filter serialization, LRU eviction, automatic flush, restart persistence, cross-tier scans, newest-value precedence, tombstone masking, and compaction cleanup.

Measured performance

Criterion microbenchmarks were run in release mode on a local Windows development machine. These numbers describe this build and dataset; they are directional engineering measurements, not production service-level claims.

OPERATION	WORKLOAD	ESTIMATED LATENCY
Sequential put	Unique key and value, WAL sync on each write	529.42 us
MemTable hit	Point lookup among 500 in-memory keys	215.17 ns
Cached SSTable hit	Point lookup among 1,000 persisted keys	436.41 ns
Bloom-filter miss	Absent key rejected before block access	211.40 ns
Range scan	100 keys across multiple SSTables	166.56 us
Full scan	2,000 keys across multiple SSTables	537.21 us

Next engineering steps

The next version would add WAL rotation tied to durable flush metadata, group commit, background flush scheduling, and leveled compaction. I would then benchmark mixed read-write workloads, cold-cache reads, concurrent clients, write amplification, and recovery from deliberately truncated WAL and SSTable files.

What this project demonstrates

This project gave me hands-on experience defining binary formats, reasoning about crash consistency, coordinating shared state, and testing algorithms across memory and disk. It also taught me to evaluate storage changes through their effects on durability, latency, memory use, and implementation complexity.